

OTIMIZAÇÃO DE ITINERÁRIOS: ESTRATÉGIAS PARA A GESTÃO EFICIENTE DE FROTAS VEICULARES

ROUTE OPTIMIZATION: STRATEGIES FOR EFFICIENT FLEET MANAGEMENT

CLEVERTON DOS SANTOS LILTK

Graduando em Ciência da Computação

clevertonsantos100@gmail.com

GUSTAVO DE OLIVEIRA CHRIST

Graduando em Ciência da Computação

gustavo.olicrist@gmail.com

ELVIRA PADUA LOVATTE

Profª. Msc. Centro de Pós-graduação FAESA

elvira.lovatte@faesa.br

RESUMO

Em uma empresa cuja atividade principal é o transporte, a eficiência das rotas é fundamental e representa um dos maiores desafios para os seus gestores. A dificuldade surge, pois para determinar rotas eficazes, não basta escolher o caminho mais curto — é preciso considerar fatores como: tráfego, condições das estradas, janelas de entrega, capacidade e quantidade de veículos. Os modelos apresentados neste projeto poderão ajudar na determinação de rotas satisfatórias. Para desenvolvê-los foram utilizadas GRASP e busca local adaptada visando a redução de custos e eficiência operacional. A avaliação dos modelos foi realizada com base em testes e análise dos resultados de métricas, como distância total, tempo de execução, janela de tempo e espaço ocioso total.

Palavras-chave: Meta Heurística, Algoritmos Gulosos, Pesquisa Operacional.

ABSTRACT

In a company whose primary activity is transportation, route efficiency is fundamental and represents one of the greatest challenges for its managers. The difficulty arises because to determine efficient routes, it is not enough to choose the shortest path - it is necessary to consider factors such as: traffic, road conditions, delivery windows, vehicle capacity, and quantity. The models presented in this project can help in determining satisfactory routes. To develop them, GRASP and adapted local search were used aiming at cost reduction and operational efficiency. The evaluation of the models was carried out based on tests and analysis of the results of metrics such as total distance, execution time, time window, and total idle space.

Keywords: Meta-heuristics. Greedy Algorithms. VRP.

INTRODUÇÃO

A logística é o processo de planejamento, implantação e controle de fluxos de mercadorias, serviços e informações desde o ponto de origem até o ponto de destino. Entre suas principais atividades, está a gestão de frotas, que consiste na administração de um conjunto de veículos pertencentes a uma organização. Esse processo exige um conhecimento abrangente por parte do gestor, já que visa reduzir custos, minimizar riscos e aumentar a eficiência das operações realizadas.

A gestão de frotas envolve desafios significativos relacionados ao gerenciamento de custos, sendo um dos principais obstáculos enfrentados por empresas de logística no Brasil. De acordo com o Panorama de transporte rodoviário de cargas disponibilizado pelo Serviço de Apoio às Micro e Pequenas Empresas de Santa Catarina (SEBRAE-SC), em 2021, ocorreu um aumento de 30,9% no custo operacional de transporte. Esse aumento está associado à alta dos preços dos combustíveis e ao desgaste frequente dos veículos, decorrente das condições da malha.

Avaliando as demandas relacionadas à logística, surge o seguinte questionamento: é possível otimizar o planejamento do itinerário de frotas veiculares? Com base nesse problema, formula-se a seguinte hipótese: a diminuição de custos operacionais de frotas veiculares e seus itinerários pode ser realizada por meio da implementação de um algoritmo capaz de determinar rotas viáveis em um tempo aceitável. Além disso, o projeto visa desenvolver uma solução algorítmica capaz de calcular de forma eficiente o itinerário ideal para uma frota de veículos.

O cenário logístico no Brasil apresenta um modal predominantemente rodoviário, ou seja, o fluxo e escoamento de bens e serviços é realizado majoritariamente nas rodovias. Dessa forma, determinar rotas rodoviárias que sejam economicamente viáveis é um problema enfrentado por muitas empresas. A expansão e manutenção da malha rodoviária também podem reduzir as dificuldades que as empresas de transporte enfrentam. Este projeto é relevante pois pode contribuir para aprimorar o planejamento logístico empresarial e reduzir custos operacionais para o setor, visto que o mesmo é importante para o desenvolvimento econômico e comercial. Como resultado, espera-se contribuir com o planejamento logístico e desenvolvimento de métodos eficientes para redução de custos operacionais de empresas e organizações.

O PROBLEMA DE ROTEAMENTO DE VEÍCULOS

Conforme Manguino (2020), O Problema de roteamento de veículos (VRP) clássico (Vehicle Routing Problem ou VRP, em inglês) consiste em encontrar um conjunto de rotas, a partir de um ponto inicial, utilizando veículos idênticos, com o objetivo de suprir as demandas com o menor custo possível. Desde a sua primeira formulação surgiram variações do problema, que visam aproximar-se da realidade percebida nas atividades industriais e logísticas, incluindo diferentes tipos de restrições e características, sendo agrupados então sob o nome de *Multi Attribute Vehicle Routing Problem* (MAVRP).

Segundo Subramanian et al. (2013), desde a sua primeira formulação, o VRP tem sido abordado nas áreas de Pesquisa Operacional e Otimização Combinatória, visto que os métodos para a resolução deste problema mostraram-se eficientes quando implementadas em situações reais, entregando resultados positivos para empresas que aplicam estes métodos.

Na versão clássica do VRP, tem-se um único depósito, uma frota de veículos homogênea (veículos com o mesmo comportamento, sem custo associados ou perda de generalidade) e somente um tipo de demanda (entrega ou coleta) para todos os clientes. O objetivo é definir um conjunto de rotas para os veículos de forma que:

- i) todas as rotas sejam circulares, ou seja, comecem e terminem em um ponto, no caso, o depósito;
- ii) as demandas de todos os clientes sejam atendidas pelos veículos;
- iii) cada cliente seja atendido por um único veículo;
- iv) a capacidade de carga dos veículos não deve ser excedida;
- v) a soma total dos custos das operações de transporte deve ser minimizada. (SUBRAMANIAN et al., 2013).

O VRP é classificado como um problema Não-determinístico Polinomial Difícil (*NP-Hard*). Isso implica que não há algoritmo determinístico capaz de solucionar o problema eficientemente. Portanto, a busca de uma solução ótima para instâncias grandes do problema é

inviável em tempo computacional razoável. Por não haver um algoritmo de complexidade polinomial, procedimentos heurísticos lógicos são utilizados para definir soluções aproximadas do problema (MANGUINO, 2020).

A utilização de heurísticas, especialmente de meta-heurísticas, no desenvolvimento de soluções para o Problema de Roteamento de Veículos trouxe avanços significativos para os campos de Pesquisa Operacional e Otimização Combinatória. Esses métodos combinam a capacidade de explorar amplamente o espaço de soluções com a habilidade de refinar as melhores opções encontradas, buscando sempre soluções de alta qualidade em prazos viáveis.

O principal marco para a resolução do VRP com heurísticas foi a Utilização da Heurística das Economias (Savings Heuristics). Este método ainda é utilizado devido à sua flexibilidade em resolver instâncias do problema com inclusão de outras restrições e em permitir novas adaptações para a resolução do problema. Conforme Ballou, (2007), este método:

“...tem atravessado os anos como algo dotado da flexibilidade suficiente para resolver uma ampla coleção de restrições práticas, sendo relativamente rápido, em termos computacionais, para problemas com um número moderado de paradas, e capaz de gerar soluções que são quase ótimas.”

Devido a estas características, a Heurística das Economias tornou-se base para a resolução não apenas do VRP Clássico, mas também de suas variações com múltiplos atributos. Com isso, vários métodos resolutivos foram aplicados para solucionar os MAVRP's. Dentre os principais, Manguino (2020) cita meta-heurísticas baseadas em busca local, como Busca Tabu (*Tabu Search*), *Simulated Annealing*, Busca em Vizinhança variável (*Variable Neighbourhood Search*), *Iterated Local Search*; métodos *multi-start*, como *Greedy Randomized Adaptive Search Procedure* (GRASP); e métodos de busca populacional, como Algoritmos Genéticos e Algoritmos Evolucionários.

PRINCÍPIOS PARA UMA BOA ROTEIRIZAÇÃO E PROGRAMAÇÃO

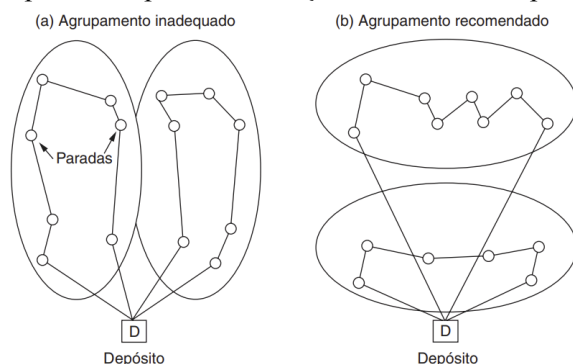
Devido à complexidade do VRP, uma determinada entrada de dados pode gerar um grande número de soluções possíveis. Portanto, é essencial avaliar se as rotas geradas pelos algoritmos representam boas soluções ou não. Ballou (2007) estabelece oito princípios para uma boa roteirização e programação de veículos. Segundo o autor, princípios como esses dão condições de

encontrar soluções satisfatórias, ainda que não necessariamente ótimas, para problemas realistas de roteirização e programação. Estes princípios são:

1. **Carregar veículos com volumes destinados a paradas que estejam mais próximas entre si**, a fim de minimizar o tráfego e tempo total de trânsito;
2. **Paradas em dias diferentes devem ser combinadas para produzir agrupamentos concentrados**: quando houver, devem ser segmentadas em problemas de roteirização e programação diferentes para cada dia da semana.
3. **Comece os roteiros a partir da parada mais distante do depósito**: Uma vez identificada a parada mais distante, selecione os pontos próximos a esta parada para compor a rota;
4. **O sequenciamento das paradas num roteiro deve ter forma de lágrima**, Ou seja, um sequenciamento não deve possuir nenhum tipo de superposição, exceto se houver algum tipo de restrição de janela de tempo e a imposição de fazer paradas para coletas depois das entregas;
5. **Os roteiros mais eficientes são aqueles que fazem uso dos maiores veículos disponíveis**: a utilização de um veículo com capacidade suficiente para abastecer todas as paradas de um roteiro minimizaria sempre a distância ou tempo total percorrido para atender todos os pontos;
6. **A coleta deve ser combinada nas rotas de entrega em vez de reservada para o final dos roteiros**. Isso diminui o número de superposições de roteiros;
7. **Uma parada removível de um agrupamento de rota é uma boa candidata a um meio alternativo de entrega**: Caso haja uma parada distante de um agrupamento de pontos, a utilização de veículos menores para atender pode revelar-se mais econômica, dependendo da distância e volume;
8. **As pequenas janelas de tempo de paradas devem ser evitadas**: As restrições das janelas de tempo de paradas muito pequenas podem forçar uma sequência de paradas longe do padrão ideal. Portanto, qualquer parada não adequada deve ter seus limites de janela de tempo renegociados e ampliados, quando possível.

A figura 1 (a) exemplifica um agrupamento inadequado de pontos de entrega, visto que os pontos mais distantes do depósito poderiam ser atendidos por um veículo apenas, não necessitando de viagens longas de dois veículos. Por outro lado, a figura 1 (b) mostra um agrupamento recomendado, em que os pontos mais distantes do depósito estão atendidos em uma única rota.

Figura 1: Agrupamentos para a destinação de volumes de paradas a veículos.



Fonte: Ballou (2007)

ALGORITMOS GULOSOS

Algoritmo guloso é uma técnica de projeto de algoritmos que obtém uma solução seguindo uma sequência de escolhas. Para cada ponto de decisão, o algoritmo escolhe a melhor opção momentânea. Conforme Cormen et al. (3a ed. 2012), uma solução gulosa pode ser proposta para um determinado problema se ele possuir as seguintes propriedades:

1. **Propriedade de escolha gulosa:** pode-se montar uma solução globalmente ótima fazendo escolhas locais ótimas, ou seja, para cada ponto de decisão, a escolha deve ser feita sem considerar resultados de subproblemas, e sim para o problema em questão;
2. **Subestrutura ótima:** Um problema exibe subestrutura ótima se uma solução ótima para o problema contiver soluções ótimas para subproblemas.

Estes algoritmos possuem características que emergem como alternativas eficazes para encontrar soluções de boa qualidade em tempo computacional aceitável, trazendo resultados próximos ao ótimo (CORMEN, 2012).

Os algoritmos gulosos são semelhantes aos algoritmos de programação dinâmica, pois ambos exploram a subestrutura ótima para encontrar soluções. No entanto, a programação dinâmica aborda o problema de forma mais sistemática, encontrando primeiro soluções ótimas para subproblemas e depois fazendo uma escolha informada. Em contraste, os algoritmos gulosos fazem uma escolha ótima inicial e, em seguida, resolvem o problema resultante sem se preocupar com a otimização de todos os subproblemas. A garantia de uma solução ótima por um algoritmo guloso depende das características do problema e da escolha da heurística, além de que, para problemas com muitas restrições interdependentes, uma escolha gulosa inicial pode levar a um beco sem saída, impedindo o algoritmo de encontrar uma solução viável. (CORMEN et al, 2012).

Ao longo da história, várias heurísticas baseadas em algoritmos gulosos foram aplicadas para a resolução do VRP. Toth e Vigo (2001, p. 1-27) descrevem a Heurística das Economias de Clarke e Wright como uma abordagem gulosa para a resolução do problema. Segundo os autores, a pesquisa de Clarke e Wright, juntamente com os estudos de Dantzig e Ramser, contribuíram para a proposição de muitos modelos e algoritmos exatos e heurísticos para a solução ótima e aproximada das diferentes versões do VRP.

TECNOLOGIAS UTILIZADAS

Javascript e Interpretadores Javascript

Conforme a MDN Web Docs, a linguagem de programação JavaScript (comumente abreviada para JS) é uma linguagem de programação interpretada, de tipagem dinâmica, multiparadigma (suportando os estilos orientação a objetos, imperativo e funcional), baseada em protótipos e objetos com funções de primeira classe, conhecida por ser uma linguagem *script* para páginas *web*. A linguagem de programação foi escolhida para o projeto devido as seguintes características:

- **Programação assíncrona:** a linguagem é otimizada para esta técnica de programação, ao utilizar uma estrutura nativa chamada Promises;
- **Funções de primeira classe:** Ao tratar as funções como variáveis, a linguagem permite refatorar os códigos-fonte com agilidade e modularizar os programas

facilmente;

- **Métodos nativos para vetores (*Arrays*):** A linguagem possui métodos de busca, ordenação e iteração em vetores nativamente implementados (*sort*, *indexOf*, *find*, *some*, *every*, entre outros).

Por ser uma linguagem construída para navegadores *web*, seu uso fora destes ambientes só é possível com o uso um ambiente de execução (*runtime*) para que os códigos JS sejam executados utilizando um motor de interpretação (*engine*). Para o projeto, foram utilizados dois *runtimes* para executar os algoritmos:

- **NodeJS:** *runtime* de código aberto e multiplataforma, executando a *engine* V8 do navegador *web* Google Chrome;
- **Bun:** *runtime* de código aberto, que utiliza a *engine* JavaScriptCore, utilizado em navegadores WebKit como Apple Safari.

OpenStreetMap

A OpenStreetMap é uma iniciativa para criar e fornecer dados geográficos gratuitos. A Fundação OpenStreetMap é uma organização internacional sem fins lucrativos que apoia, mas não controla, o Projeto OpenStreetMap e dedica-se a incentivar o crescimento, desenvolvimento e distribuição de dados geoespaciais gratuitos e a fornecer os mesmos para quaisquer usos.

A fundação apoia diversas ferramentas baseadas em software, dentre as quais foram utilizadas para o projeto:

- **Leaflet:** Uma biblioteca Javascript de código aberto para desenvolvimento de mapas interativos em navegadores, com recursos e módulos de visualização de elementos HTML nos mapas. Essa biblioteca foi utilizada para validação visual dos algoritmos desenvolvidos neste projeto, permitindo a visualização dos pontos de entrega de cada solução, sua ordem de visitação e o trajeto a percorrer da rota;
- **Open Source Routing Machine (OSRM):** Um software de código aberto utilizado para calcular rotas entre dois pontos, retornando dados de distância, tempo e restrições como vias de mão única, vias para pedestres, veículos comerciais, entre outras. A

OpenStreetMap possui uma *Application Programming Interface* (API), utilizada neste projeto para consultar os dados de distância e tempo de deslocamento entre dois pontos, essenciais para desenvolver e avaliar a solução do projeto.

GERAÇÃO DE INSTÂNCIAS

Para avaliar o desempenho dos algoritmos construídos no projeto, foram feitos testes computacionais. Para o projeto, a geração de instâncias foi inspirada na proposta por Manguino (2020). O autor gerou instâncias adaptadas para a resolução do Problema de Roteamento de Veículos com Frota Heterogênea, Janelas de Tempo e Custos Escalonados, com 100 pontos de entrega cada, seguindo critérios de distribuição espacial, janelas de tempo e custo de veículos.

Janelas de Tempo

Para cada ponto de entrega i do conjunto S , há uma janela de tempo definida no intervalo $[s_i, e_i]$, em que $s_i < e_i$. Esses valores definem o intervalo de tempo em que um ponto de entrega está disponível para ser atendido, em que s_i é o início e e_i o fim da janela de tempo. Os atendimentos não podem acontecer nem antes e nem depois deste intervalo. Não há uma janela de tempo para o depósito, visto que o mesmo não possui nenhum serviço a ser executado.

Manguino (2020) considerou em seu trabalho que, caso um veículo chegue a um determinado ponto j , partindo do ponto i , antes do início da janela, o mesmo deve aguardar até a disponibilidade do atendimento.

Características das instâncias

Para o projeto, foram geradas 10 instâncias para cada modelo desenvolvido no projeto, em que cada uma delas possui 270 pontos de entrega e um depósito, escolhidas no mapa seguindo alguns critérios:

i) Distribuição espacial: Os pontos de entrega foram distribuídos de forma que houvesse pontos aninhados em grupos, ou seja, próximos entre si, e distribuídos aleatoriamente no

Após os testes, os dados de tempo de execução, distância total das rotas geradas e espaço ocioso dos veículos foram coletados para fazer uma análise de desempenho dos modelos construídos.

CRIAÇÃO DA MATRIZ DE DADOS

Assis et al. (2013) afirma que os problemas de roteamento de veículos podem ser representados por grafos, o que facilita a visualização e a análise das rotas possíveis. Neste trabalho, foi utilizado um grafo valorado com $n + 1$ vértices: n vértices que representam os pontos de entrega e um vértice adicional que representa o depósito, e cada aresta possui um valor associado, indicando o **custo** (ou distância) de percorrer aquele caminho específico. Modelar o VRP como um grafo valorado facilita a inclusão de novas restrições e a adaptação a características específicas do problema, como capacidade dos veículos e janelas de tempo. Essa estrutura em grafos é, portanto, uma ferramenta fundamental para o desenvolvimento de soluções no campo da otimização de rotas.

Para armazenar as informações contidas no grafo, foi utilizada uma matriz quadrada $A = (a_{ij})_{n+1,n+1}$. A cada linha e coluna de A foi associado um ponto de entrega ou vértice, de modo que o elemento a_{ij} , localizado na linha i e coluna j , representa o custo do caminho c_{ij} do ponto i até o ponto j .

Em inúmeras situações o custo de um caminho c_{ij} é determinado com base na distância e duração da rota do ponto i ao ponto j . Neste trabalho, estes custos foram obtidos por meio de consultas à API do *Open Source Routing Machine* (OSRM) utilizando as coordenadas geográficas dos pontos i e j . Quando $i = j$, os valores dos custos são zero. Nos casos em que não há caminho entre i e j , assegurando que, nas validações de janelas de tempo, distância e capacidade do veículo, o trajeto entre i e j seja desconsiderado.

Dessa forma, cada elemento a_{ij} da matriz A pode ser determinado genericamente por:

$$a_{ij} = \{\text{custo de } c_{ij} ; \text{ caso exista caminho de } i \text{ para } j \neq 0 ; \text{ caso } i = j \infty ; \text{ caso contrário}\}$$

Após a geração da matriz A , ela foi armazenada em um arquivo na Notação de Objeto Javascript (JSON), permitindo que seu conteúdo fosse facilmente carregado nos programas desenvolvidos para consultar os valores armazenados. A construção e armazenamento desta estrutura de dados foram fundamentais para o desenvolvimento do projeto, com o intuito de evitar bloqueios de requisição à API do OSRM, garantindo eficiência no acesso às informações de roteamento.

HEURÍSTICA GULOSA SIMPLES

A utilização de heurísticas gulosas para a resolução do problema está relacionada aos princípios e boas práticas de roteirização estabelecidos anteriormente.

Neste trabalho, a estratégia gulosa foi utilizada durante a escolha de cada ponto de parada inserido em uma rota R_i . A escolha foi realizada observando-se entre os pontos de parada que ainda não pertenciam a nenhuma rota, aquele mais próximo do último ponto incluído na rota R_i . Essa abordagem foi adotada com o objetivo de agrupar, em uma mesma rota pontos de paradas com localizações próximas, o que tende a produzir trajetos mais eficientes em termos de tempo de viagem e distância percorrida.

Esta estratégia se assemelha à descrita por Gonçalves et al (2013 apud Goldberg, Luna 2000) com o algoritmo de inserção do ponto mais próximo. Esta heurística possui um processo de três níveis de decisão. A escolha do vértice a ser inserido, a posição de inserção e a decisão de um ciclo inicial. Gonçalves et al:

“Normalmente este tipo de heurística parte de uma sub-rota (*subtour*) inicial (um ciclo normalmente de comprimento igual a três) e vai selecionando e inserindo vértices ainda não incluídos na solução até completar o ciclo”

Ainda na escolha do vértice k a ser inserido em uma determinada rota considerou-se também respeitar a capacidade do veículo e janela de tempo do ponto de parada e de janelas de tempo.

Dessa forma, para que o ponto k seja selecionado para inclusão em uma determinada rota específica, devem ser atendidas as seguintes condições:

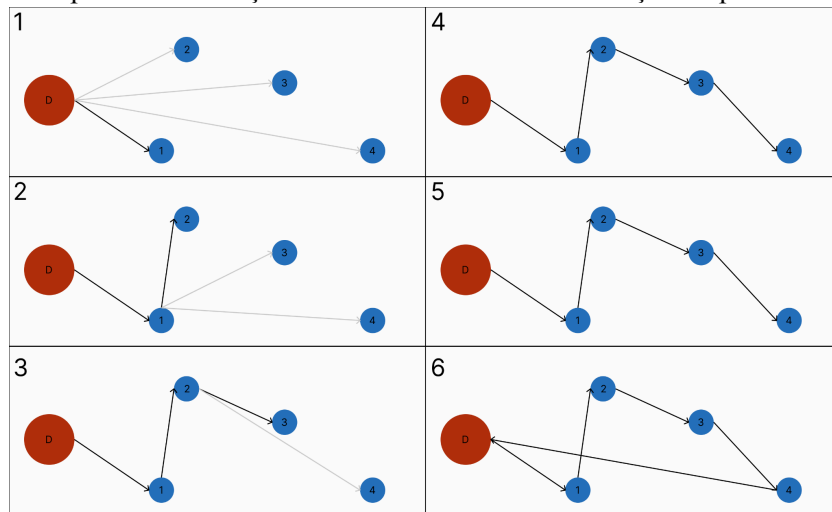
- k deve apresentar a menor distância em relação à última parada (ou seja, o último ponto da rota em análise);

- o instante de chegada ao ponto k deve respeitar a janela de tempo específica para este ponto;
- a demanda associada ao ponto de parada k somada a demanda total acumulada da rota, não deve exceder a capacidade total do veículo designado para esta rota.

Se o ponto k satisfizer todas essas condições, ele é adicionado à rota após a última parada. Caso não exista um ponto k que cumpra os critérios estabelecidos, a rota é encerrada, e o veículo deve retornar ao depósito após a última parada."

A figura 3 ilustra como ocorre a escolha de um vértice k , considerando apenas a menor distância entre k e o último ponto de parada da rota. É possível verificar na figura 3(1) que a partir do depósito (vermelho), procura-se o ponto mais próximo para roteirizar. As distâncias estão representadas pelas arestas que estão em cinza, sendo a cinza escuro a menor de todas. Na figura 3(2), tem-se novamente 3 arestas conectadas ao último ponto da sub rota, sendo uma delas mais destacada por ser a menor. Assim, mais um ponto de parada é incluído na rota. Este processo se repete até a figura 3(5), em que todos os pontos de parada já foram incluídos na rota. A figura 3(6) destaca o trecho final da rota em que o veículo retorna para o depósito.

Figura 3: Exemplo de roteirização baseado na heurística de seleção do ponto mais próximo.



Fonte: elaborado pelos autores (2024).

A abordagem gulosa de selecionar o ponto mais próximo da última parada e adicioná-lo em

seguida é eficiente em tempo computacional, mas reduz o espaço de busca de soluções possíveis a serem analisadas, o que pode direcionar o algoritmo a definir uma solução subótima ou, em alguns casos, a uma solução de baixa qualidade.

DESENVOLVIMENTO DE HEURÍSTICA GRASP

Segundo Resende e Silva (2013), o GRASP é um algoritmo de busca gulosa que consiste em iniciar uma busca local a partir de várias soluções geradas aleatoriamente, com a expectativa de que haja um caminho de melhoria a partir de uma destas soluções até a solução ótima ou próxima da ótima. Ao fim da construção gulosa, a solução obtida é viável para diversas instâncias da heurística. Entretanto, para alguns problemas, a solução pode ser inviável, precisando, então, de um procedimento de melhoria ou reparo da solução com o intuito de restaurar a sua viabilidade, conforme o algoritmo a seguir:

Figura 4: Pseudocódigo de um GRASP básico para minimização.

Dados: critério de parada
Resultado: solução x^* .

```

1  $x^* \leftarrow \infty$ ;
2 enquanto critério de parada não for satisfeito faça
3    $x \leftarrow \text{GulosoAleatorio}(\cdot)$ ;
4   se  $x$  não for viável então
5      $x \leftarrow \text{reparo}(x)$ ;
6   fim
7    $x \leftarrow \text{BuscaLocal}(x)$ ;
8   se  $f(x) < f(x^*)$  então
9      $x^* \leftarrow x$ ;
10  fim
11 fim
12 retorna  $x^*$ ;
```

Fonte: Resende e Silva (2013)

Seguindo a sequência deste pseudocódigo, inicialmente é realizado um processo guloso aleatório para encontrar uma solução x para um problema de minimização. Em seguida, a solução x passa por um processo de verificação de viabilidade. Caso x não seja viável, uma função de reparo é executada para transformá-la em uma solução viável. Posteriormente, x é submetida a um processo de melhoria por meio de uma busca local. Ao final, a solução x é comparada com uma solução anterior x^* . Se x for melhor que x^* , então x^* é atualizada com o valor de x .

A função gulosa aleatória (linha 3- pseudo código apresentado na figura 4) é uma adaptação da heurística gulosa simples apresentada anteriormente. Ao invés de selecionar apenas um

vértice na iteração sobre os pontos remanescentes, são selecionados uma lista de vértices candidatos, ou seja, pontos que possuem suas restrições atendidas e não excedam a capacidade do veículo. Desta lista, é escolhido um ponto aleatoriamente para compor a rota. O processo ocorre até que não existam mais pontos remanescentes que atendam às restrições e a rota do veículo é finalizada.

Ao final da execução, o algoritmo guloso aleatório disponibiliza uma solução para o problema. Em certos casos, as soluções podem ser inviáveis ou até mesmo distantes da ótima. Como apresentado anteriormente, as versões do VRP são classificadas como *NP-hard* e o algoritmo guloso pode gerar rapidamente uma solução que pode ser muito ruim.

Busca local

Os processos de busca local, Conforme Resende e Silva (2013), partem de uma solução inicial, por sua vez tida como a solução corrente, a procura de uma solução melhor próxima da solução corrente. Caso exista, a nova solução se tornará a solução corrente e a busca local será recursivamente aplicada. O procedimento termina quando não for possível gerar uma solução melhor do que a corrente, retornando, assim, a solução corrente.

No projeto, foram implementadas estratégias de busca intra-rota (busca de soluções próximas que envolvem operações em apenas uma rota) e inter-rota (busca de soluções próximas que envolvem operações entre duas ou mais rotas). Essas abordagens foram descritas e aplicadas por Toth e Vigo (2001, p. 195-224), Subramanian et al (2013) e Manguino (2020) para pós otimização de soluções.

Para realizar as buscas intra e inter rotas foram empregadas as operações de permutação de rotas (*Swap*) e realocação de pontos em uma rota (*Relocate*). A permutação e realocação são descritas por Manguino (2020) e Subramanian et al (2013) como:

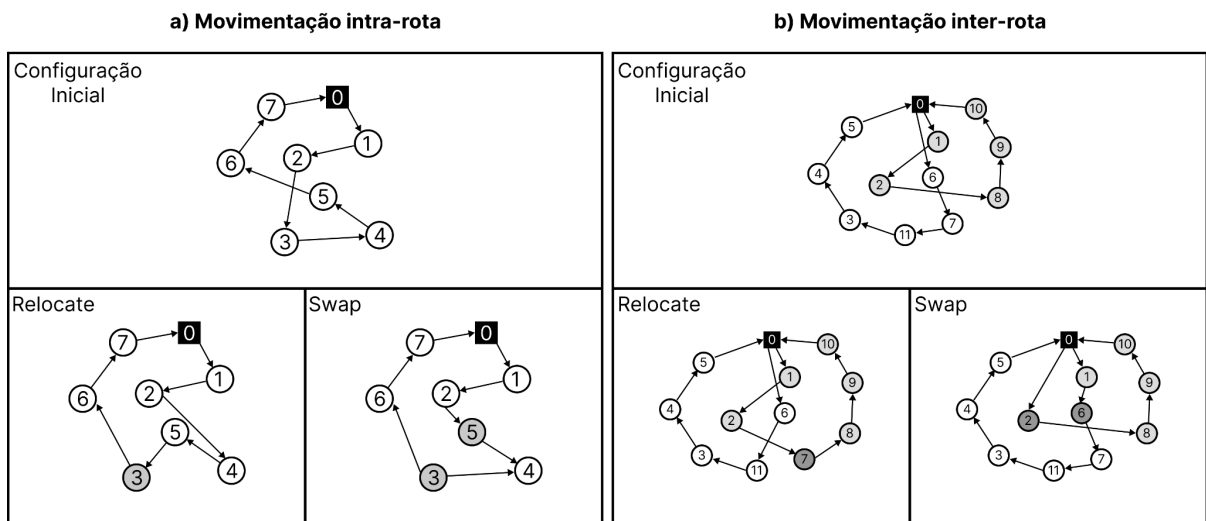
- O *Swap* fará uma troca entre um ponto i de uma rota $R1$, e um ponto j de uma rota $R2$. Se a soma das distâncias das rotas diminuir, a solução é atualizada. Esta abordagem é aplicada tanto na otimização intra-rota (onde $R1 = R2$) quanto na inter-rota;
- O *Relocate* fará uma remoção de um ponto i de uma rota $R1$ e reinserção na posição j de uma rota $R2$. Se a distância total $R1$ diminuir após a troca, a solução é atualizada.

Esta abordagem é aplicada tanto na otimização intra-rota (onde $R1 = R2$) quanto na inter-rota.

A figura 5 exemplifica visualmente como as operações de troca intra-rota e inter-rota ocorrem. Na movimentação intra-rota da figura 5 (a), o Movimento Relocate removeu o ponto 3 de sua ordem original para uma nova posição, mudando a ordem de visitação de 0,1,2,3,4,5,6,7,0 para a nova disposição 0,1,2,4,5,3,6,7,0. O movimento Swap, no entanto, fez uma troca de visitasões entre os pontos 3 e 5, fazendo com que a nova ordem seja 0,1,2,5,4,3,6,7,0.

Na movimentação inter-rota da figura 5 (b), o Movimento Relocate removeu o ponto 7 de sua rota original (pontos brancos) e reinseriu na outra rota (pontos cinza claros), enquanto que o movimento Swap fez uma permutação entre os pontos 2 e 6 de ambas as rotas.

Figura 5: Ilustração das rotas após as movimentações *SWAP* e *RELOCATE* aplicadas na otimização intra-rota e inter-rota.



Fonte: Subramanian et al (2013), adaptado.

MELHORIAS NOS MODELOS DE ROTEIRIZAÇÃO

Durante o desenvolvimento do projeto, foram realizados processos de otimização dos algoritmos para alinhar as soluções com as boas práticas de roteirização mencionadas anteriormente. As melhorias incluíram estratégias como priorizar a criação de rotas para veículos de maior capacidade, iniciar as rotas a partir do ponto mais longínquo para cada veículo e aplicar o algoritmo de busca local sobre a heurística gulosa simples.

Neste trabalho foram utilizadas diferentes abordagens de otimização e algoritmos para melhorar a eficiência da roteirização, porém nenhuma incluiu uma etapa preliminar de organização dos veículos com base em critérios específicos, tipo de carga e quantidade, ou a pré-seleção de pontos com características específicas. A ausência dessa abordagem indicou lacunas a serem exploradas, uma vez que estas estratégias podem influenciar significativamente a qualidade das rotas geradas e a eficiência do processo logístico como um todo.

Considerar veículos de maior carga para roteirizar primeiro

Seguindo o quinto princípio de boas práticas de roteirização, as primeiras rotas a serem construídas começam a partir do veículo de maior capacidade de carga. Para isso, ao gerar as rotas, é feito um processo de ordenação descendente dos veículos por capacidade antes de realizar os passos de roteirização.

Começar as rotas a partir do ponto mais longínquo

Seguindo o terceiro princípio de boas práticas de roteirização, as rotas começam a partir do ponto de maior distância em relação ao depósito. Para isso, antes de fazer as iterações de busca dos pontos mais próximos à última parada, é feita uma busca na matriz de distâncias por um ponto s cujo trajeto do depósito D até s seja máxima, respeitando as restrições de janelas de tempo e capacidade de carga do veículo. Se s existir, ele é adicionado à rota do veículo e é removido da lista de pontos remanescentes R .

Busca local para a heurística gulosa simples

Após o desenvolvimento dos algoritmos de busca local para a resolução do problema utilizando a heurística GRASP e a aplicação dos processos de melhoria anteriores, o processo de busca local também foi aplicado a heurística gulosa simples. O principal motivador foi a possibilidade de gerar sobreposição de rotas ao começar o processo pelo ponto mais distante. Com a busca local, ampliou-se o espaço de busca, resultando em soluções melhores.

Enviesamento de seleção de pontos para o GRASP

Seguindo o primeiro princípio de boas práticas de roteirização, para o modelo GRASP a etapa de seleção de pontos passou por um processo de enviesamento. O algoritmo foi modificado

para reduzir a quantidade de candidatos da lista para um valor x . Os x candidatos são selecionados pelo critério de distância, ou seja, são os x pontos mais próximos da última parada. Após este processo, a heurística segue a sua execução normalmente.

Considerando todas as heurísticas e modificações desenvolvidas, foram implementados 6 modelos para resolver uma mesma instância do problema de roteamento. A análise dos resultados obtidos por uma ou mais versões auxiliou na reformulação e aprimoramento de novas versões do modelo.

Foram desenvolvidas diferentes versões dos modelos, denominadas como: **HGS** e **GRASP** com e sem modificações. Quando a versão clássica possui alguma modificação, acrescenta-se um dos sufixos l, v, b ou k ao nome principal. A seguir, apresenta-se uma descrição sucinta das versões.

- **HGS**: Implementação de Heurística Gulosa Simples, sem modificações;
- **GRASP**: Implementação de um modelo GRASP, sem modificações;
- **-l**: Sufixo que indica que o modelo constrói as rotas a partir do ponto mais distante em relação ao depósito;
- **-v**: Sufixo que indica que o modelo realiza a ordenação decrescente dos veículos por capacidade antes da roteirização;
- **-b**: Sufixo que indica que o modelo utiliza um processo de busca local, aplicável apenas no modelos HGS;
- **-k**: Sufixo que indica que o modelo possui enviesamento de seleção de pontos vizinhos. Aplicável apenas nos modelos GRASP.

As versões desenvolvidas retornam soluções literais e gráficas **formato literal** e **gráfico**, facilitando tanto a análise visual das rotas geradas quanto a avaliação dos resultados numéricos.

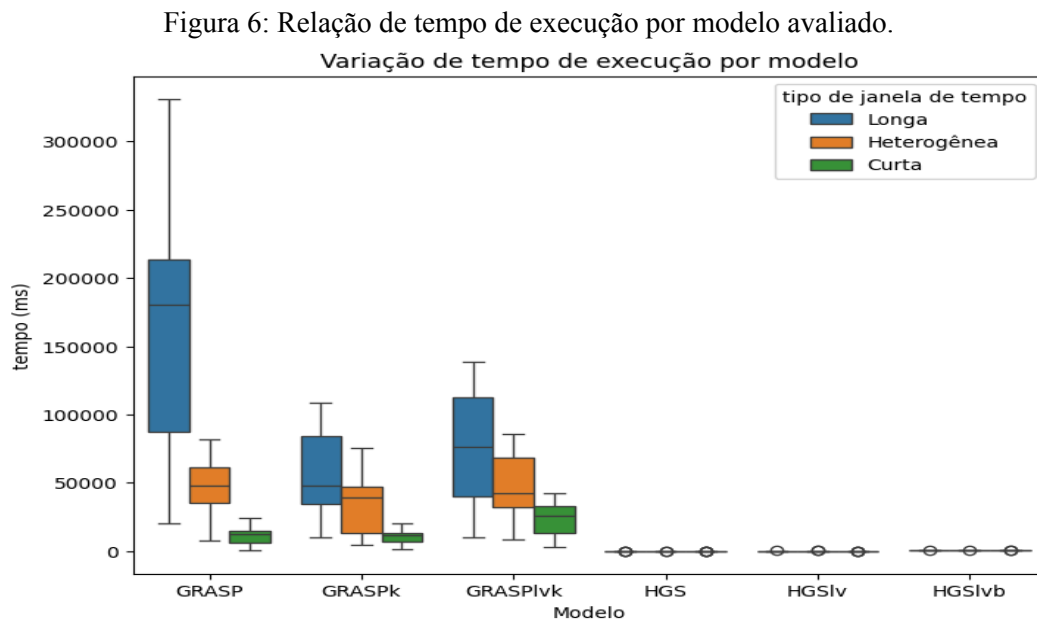
RESULTADOS E DISCUSSÃO

Após a execução dos testes computacionais, foram realizadas avaliações estatísticas e visuais das soluções geradas pelos modelos, visando uma análise aprofundada de seu desempenho e qualidade. As métricas analisadas para medir a performance de uma solução foram tempo de

execução e custo da rota. Os resultados dessas medidas são detalhados a seguir.

Tempos de Execução

Os modelos baseados no GRASP apresentaram uma maior variabilidade nos tempos de execução, com valores de mediana e quartis superiores, o que indica que, em média, esses modelos demandaram mais tempo para finalizar suas tarefas. Em contraste, os modelos baseados em HGS exibiram tempos de execução consideravelmente menores, sugerindo um desempenho superior em termos de eficiência temporal, como ilustrado pelo gráfico na Figura 6.



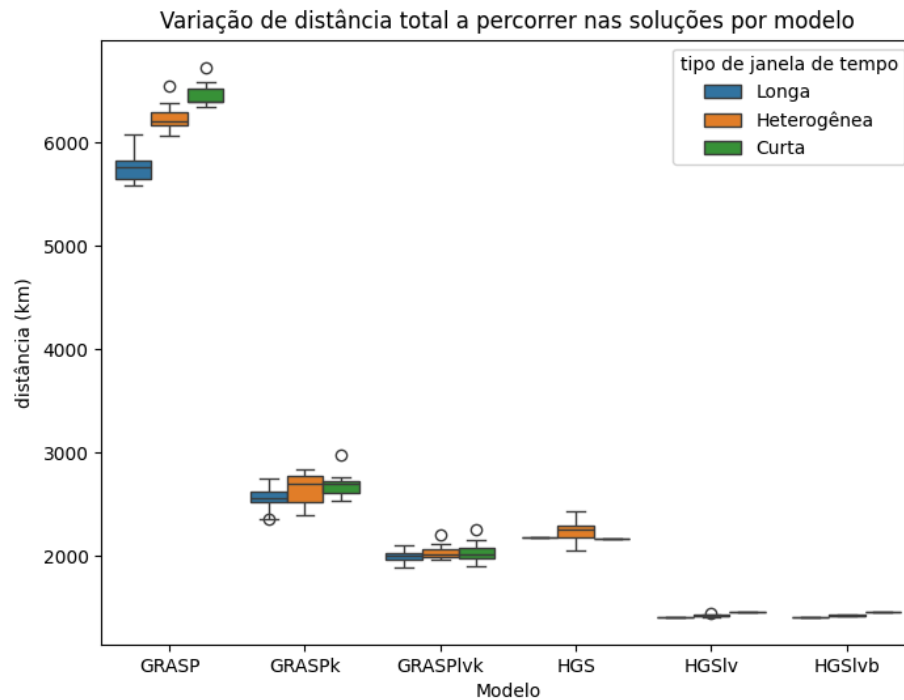
Fonte: elaborado pelos autores (2024)

Observou-se que os modelos GRASP com modificações apresentaram intervalos menores e mais estreitos nos tempos de execução em comparação com o GRASP sem modificações. Isso sugere que as modificações implementadas contribuíram para uma melhora de performance no tempo de execução desses modelos. Ao analisar os modelos HGS isoladamente, nota-se que os três modelos possuem uma faixa relativamente estreita de tempos de execução, sugerindo consistência nos tempos de resposta. Contudo, o modelo HGSlvb apresenta tempos de execução ligeiramente superiores aos dos demais modelos HGS.

Custo da rota

Analisando as soluções quanto ao tamanho da rota, percebe-se que os modelos GRASP modificados obtiveram soluções melhores comparadas ao GRASP sem modificações. O modelo GRASPlvk conseguiu alcançar soluções com distância total ligeiramente menor em comparação com o modelo HGS. Os modelos HGSlv e HGSlvb alcançaram as menores distâncias totais entre os modelos. As melhorias descritas anteriormente e aplicadas aos modelos geraram resultados positivos quanto a esta métrica. Todavia, o processo de busca local no modelo HGSlvb não conseguiu melhorar a solução final durante a sua execução, como ilustrado pelo gráfico da Figura 7.

Figura 7: Variação de distância total para cada modelo avaliado.



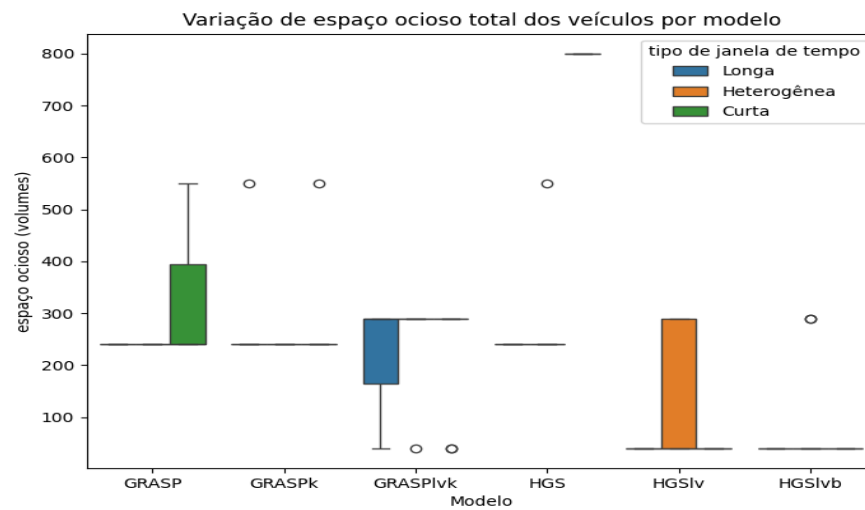
Fonte: elaborado pelos autores (2024).

Ao verificar o espaço ocioso total, ou seja, a soma da capacidade não aproveitada dos veículos roteirizados das soluções, percebeu-se que as soluções geradas pelos modelos GRASP modificados tiveram médias muito próximas e baixa variabilidade em suas soluções. Os modelos HGS tiveram médias mais constantes e espaço ocioso total menores comparados aos modelos HGS. Os modelos HGS modificados foram mais performáticos nesta métrica comparados aos modelos GRASP e HGS sem modificações, visto que suas métricas foram melhores e mais constantes por possuírem um desvio padrão baixo e médias menores, como

visto no gráfico da figura 8.

O modelo HGS não se mostrou performático na instância de janelas de tempo curtas com relação à métrica de espaço ocioso total. Conforme descrito anteriormente, algoritmos gulosos podem alcançar soluções inviáveis, dependendo do conjunto de restrições. Isso indica que, para janelas de tempo curtas, um modelo guloso simples não seja eficiente no quesito de ociosidade de espaço.

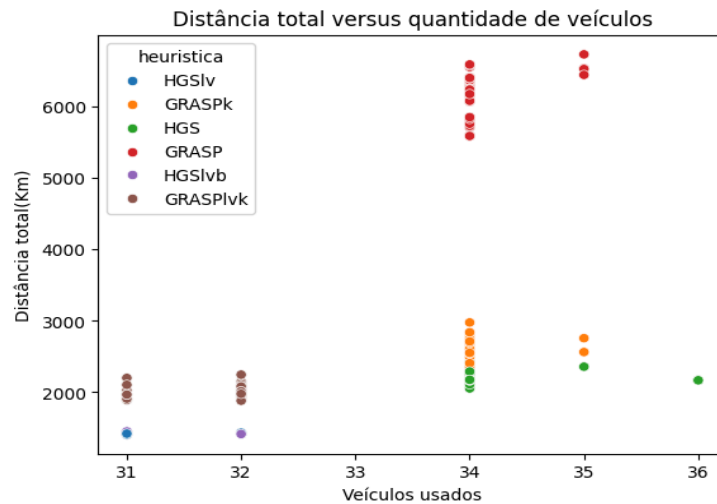
Figura 8: Variação de espaço ocioso total para cada modelo avaliado.



Fonte: elaborado pelos autores (2024).

Analisando a dispersão entre a distância total e quantidade de veículos utilizados por modelo no gráfico da figura 9, percebe-se que a ordenação prévia dos veículos e a seleção forçada do ponto mais longe em relação ao depósito geraram soluções melhores, utilizando menos veículos e obtendo uma distância total menor em comparação aos modelos que não possuem essas modificações, indicando uma melhoria no processo de roteirização.

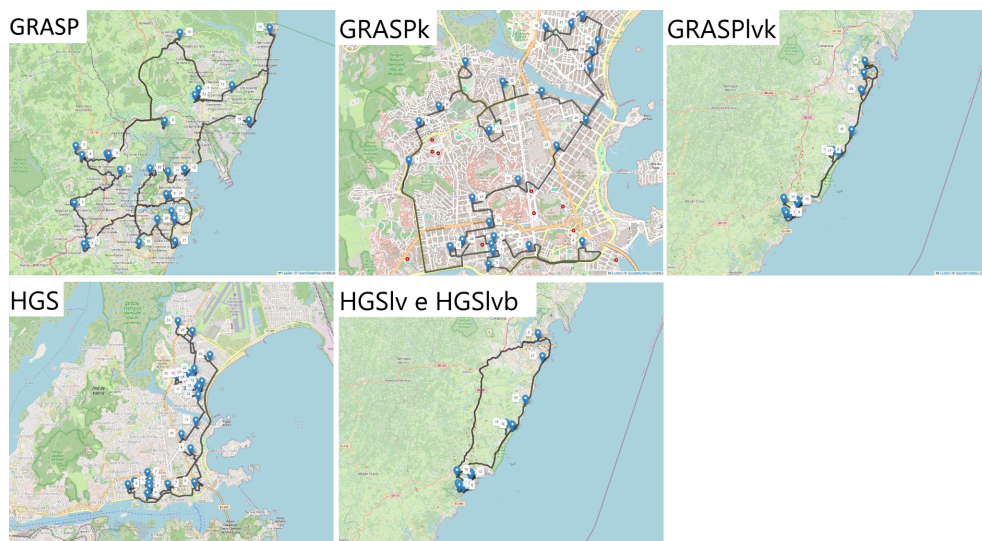
Figura 9: Dispersão de distância total para quantidade de veículos utilizados.



Fonte: elaborado pelos autores (2024)

Ao fazer a análise visual das rotas geradas pelos modelos, percebeu-se que elas tinham um formato próximo ao de uma lágrima, conforme descrito nas boas práticas de roteirização. Entretanto, todas as soluções apresentavam sobreposições em determinados trechos. Como mencionado, ao trabalhar com janelas de tempo, as sobreposições podem ser comuns nas rotas das soluções. A seguir, um conjunto de imagens mostrando rotas geradas para um veículo por diferentes modelos, sendo exibidas com o auxílio da biblioteca Leaflet.

Figura 10: Exemplos de soluções geradas para um determinado veículo em diferentes modelos.



Fonte: elaborado pelos autores (2024)

CONCLUSÃO

Este projeto teve como principal objetivo propor melhorias no planejamento de itinerários veiculares por meio de uma solução algorítmica. Para tanto, foram realizadas pesquisas bibliográficas acerca do problema de roteamento de veículos, suas características, variações, soluções propostas e boas práticas no âmbito operacional. Essa pesquisa permitiu estabelecer uma relação entre algoritmos gulosos e a resolução do problema de roteamento, resultando no desenvolvimento de modelos heurísticos para abordar o problema de forma eficiente.

Foram elaboradas duas heurísticas para a resolução do problema de roteamento de veículos com janelas de tempo: um utilizando uma construção gulosa simples e outro baseado na heurística GRASP. Esses modelos foram submetidos a processos de modificação e avaliação de desempenho, considerando tanto o tempo de execução quanto a eficiência na resolução do problema. Os testes mostraram que essas modificações contribuíram para a obtenção de soluções de alta qualidade, com destaque para o modelo guloso simples, que se mostrou particularmente eficaz em termos de qualidade das soluções geradas.

Como proposta para o futuro, considera-se a inclusão de um processo de resolução conjunta dos problemas de roteirização e escalonamento, buscando uma abordagem mais integrada para a otimização de itinerários. Isso ampliaria a abrangência do modelo, permitindo otimizar não só as rotas, mas também o planejamento temporal das entregas, melhorando a eficiência operacional e o atendimento às demandas de maneira mais completa.

REFERÊNCIAS

ASSIS, L. et al. Problema de roteamento de veículos Multiobjetivo com coleta seletiva. *In*: LOPES, H. S.; RODRIGUES, L. C. A.; STEINER, M. T. A, Organizador (org.). **Meta-heurísticas em pesquisa operacional**. Curitiba, PR: Omnipax, 2013. 181–202.

LEAFLET. **Leaflet** - *an open-source JavaScript library for interactive maps*. Disponível em: <<https://leafletjs.com/>>. Acesso em: 25 set. 2024.

BALLOU, R. **Gerenciamento da cadeia de suprimentos/logística empresarial**. 5. ed. Porto Alegre: Bookman, 2007.

CORMEN, T. et al; **Algoritmos: Teoria e prática**. 3. ed. Rio de Janeiro: Elsevier, 2012.

GONÇALVES, S.; STEINER, M; SOUZA, L.; Metodologia para o PRV: um Estudo de Caso para a Distribuição de Água Mineral. *In*: LOPES, H. S; RODRIGUES, L. C. A.; STEINER, M. T. A,

Organizador (org.). **Meta-heurísticas em pesquisa operacional**. Curitiba, PR: Omnipax, 2013. 203–218.

MANGUINO J.; **Métodos Heurísticos Aplicados ao problema de roteamento de veículos com frota mista, janelas de tempo, e custos escalonados por faixa de distância**. Disponível em: <<https://www.teses.usp.br/teses/disponiveis/3/3136/tde-04092020-090739/pt-br.php>>. Acesso em: 20 maio 2024.

MDN WEB DOCS. **Sobre JavaScript**. MDN Web Docs, 2022. Disponível em: <<https://developer.mozilla.org/pt-BR/docs/conflicting/Web/JavaScript>>. Acesso em: 25 set. 2024.

OLIVEIRA, R. **A importância da gestão de frotas e treinamento de condutores na economia de combustível**. 2018. Disponível em: <https://ric.cps.sp.gov.br/bitstream/123456789/2935/1/20181S_OLIVEIRARenande_OD0409.pdf>. Acesso em: 13 mar. 2024.

OPENSTREETMAP FOUNDATION. **O OpenStreetMap fornece dados de mapa para milhares de sites, aplicativos móveis e dispositivos de hardware**. OpenStreetMap, 2024. Disponível em: <<https://www.openstreetmap.org/about>>. Acesso em: 1 out. 2024.

RESENDE, M.; SILVA, R.; GRASP: Procedimentos de busca Gulosos, Aleatórios e Adaptativos. *In*: LOPES, H. S; RODRIGUES, L. C. A.; STEINER, M. T. A, Organizador (org.). **Meta-heurísticas em pesquisa operacional**. Curitiba, PR: Omnipax, 2013. 1–21.

SEBRAE Santa Catarina. **Panorama do mercado de transporte rodoviário de cargas**. Florianópolis: SEBRAE/SC, 2022. Disponível em: <<https://www.sebrae-sc.com.br/observatorio/relatorio-de-inteligencia/panorama-do-transporte-de-cargas-rodoviario>>. Acesso em: 15 mar. 2024.

SUBRAMANIAN, A. et al. Um algoritmo heurístico baseado em *Iterated Local Search* para problemas de roteamento de veículos. *In*: LOPES, H. S; RODRIGUES, L. C. A.; STEINER, M. T. A, Organizador (org.). **Meta-heurísticas em pesquisa operacional**. Curitiba, PR: Omnipax, 2013. p. 165–180.

TOTH, P.; VIGO, D.; An Overview of Vehicle Routing Problems. *In*: _____ (org.). **The Vehicle Routing Problem**. Philadelphia, PA: SIAM, 2001. p. 1-27

TOTH, P.; VIGO, D.; VRP with Backhauls. *In*: _____ (org.). **The Vehicle Routing Problem**. Philadelphia, PA: SIAM, 2001. p. 195-224